

SLIGHTLY IMPROVED LOWER BOUNDS FOR THE SHANNON CAPACITY OF C_{15} AND C_{19} , FORMALISED IN THE BUYS–POLAK–ZUIDDAM LEAN FRAMEWORK

GRIFFIN LONG

ABSTRACT. We prove $\Theta(C_{15}) \geq 7.301635000991096$ and $\Theta(C_{19}) \geq 9.357203012726939$. The best values we could find were 7.301628695930103 and 9.357200030000796, both in the Lean repository of Buys, Polak and Zuiddam (BPZ) at commit `aa21eeb`. Our values improve on these by about $6.3 \cdot 10^{-6}$ and $3.0 \cdot 10^{-6}$. Every ingredient is BPZ’s: the base port systems in $C_n^{\boxtimes 4}$, the substitution tables, the terminal code and the Lean framework. Only the substitution schedules are new. They were found by an automated search over BPZ’s schedule space, run by the AI agent agent-afk. Both bounds are Lean 4 theorems, delivered as a patch that only adds files to BPZ’s repository. The trust base is the same as BPZ’s, including `native_decide`. One command reproduces the check from a fresh clone.

1. PREVIOUS BOUNDS

The Shannon capacity is $\Theta(G) = \sup_d \alpha(G^{\boxtimes d})^{1/d}$. For odd cycles C_n with $n \geq 7$ its value is unknown.

BPZ Lean repository [BPZ26]: <https://github.com/spectra-research/shannon-capacity-lean>, commit `aa21eeb12b75b0413d3fa9fb4208b5d0bf2c4d65` (2026-08-10), which was still HEAD on 2026-10-01. Its README table reads (columns: n , the bound, the power p , and the number of digits of α):

15	7.301628695930103	3664	3164
19	9.357200030000796	11856	11514

BPZ paper, arXiv:2607.29681 v1 (the only version). Its abstract gives “ $\Theta(C_{15}) \geq 7.301600534487\dots$, $\Theta(C_{19}) \geq 9.357192705918\dots$ ”, obtained by iterating Gao’s $\star$ -product [Gao26] of valid tuples. The repository README says that for $C_{15}, C_{19}, C_{21}, C_{23}$ it contains “improved product trees that will appear in the next arXiv version”.

Other sources. Itty, Rosin, Carstensen and Reichman [IRCR26] prove $\Theta(C_{15}) > 7.301399$. Gao’s fork of the BPZ repository has the older BPZ values. We did not find a C_{15} or C_{19} bound newer than `aa21eeb` (Section 6).

2. NEW BOUNDS

Theorem 1. $\Theta(C_{15}) \geq 7.301635000991096$ and $\Theta(C_{19}) \geq 9.357203012726939$.

Both are verbatim Lean statements:

```
theorem ShannonBounds.CapCertC15b.shannonCapacity_cycleGraph_15_ge :
  (7.301635000991096 : ℝ) ≤ shannonCapacity (SimpleGraph.cycleGraph 15)
theorem ShannonBounds.CapCertC19c.shannonCapacity_cycleGraph_19_ge :
  (9.357203012726939 : ℝ) ≤ shannonCapacity (SimpleGraph.cycleGraph 19)
```

Here `shannonCapacity` is BPZ’s definition in `Defs.lean`, unchanged, and `cycleGraph` is Mathlib’s.

n	p	digits	new bound	BPZ aa21eeb (p)	gain	BPZ v1
15	3424	2957	7.301635000991096	7.301628695930103 (3664)	$6.31 \cdot 10^{-6}$	7.301600534487
19	14800	14373	9.357203012726939	9.357200030000796 (11856)	$2.98 \cdot 10^{-6}$	9.357192705918

Each bound comes from an explicit integer M with $M \leq \alpha(C_n^{\boxtimes p})$, proved in Lean as `alpha_strongPower_ge`. The decimal is $M^{1/p}$ truncated to 15 places. Lean proves that the truncation is exact (`tight`: $a^p \leq M \cdot 10^{15p} < (a+1)^p$).

3. THE CONSTRUCTION IN BPZ'S FRAMEWORK

We follow the notation of BPZ's files `PortRealisation.lean`, `Layered.lean`, `Reindex.lean`, `Substitutions.lean` and `TerminalCodes.lean`.

Separation system. BPZ use the alphabet $\{B, N, A, D, O, H, V\}$ with the symmetric, irreflexive relation `Letter.sep`. A realisation in G assigns to each letter a an independent set P_a , such that P_a and P_b are separated whenever `sep(a, b)` holds. The weight of a is $w_a = |P_a|$.

Base. For $n \in \{15, 19\}$, BPZ give a `RichPortSystem` in $G_4 = C_n^{\boxtimes 4}$ (`BaseC15`, `BaseC19`). Its seven families form a realisation R_1 with weights (B, N, A, D, O, H, V) :

- C_{15} : (2839, 2833, 6, 3, 3, 3, 3), from the 2842-set of de Boer, Buys and Zuiddam [dBBZ24];
- C_{19} : (7664, 7661, 3, 2, 2, 2, 2), from the 7666-set of Baumert et al. [BMR+71] and Codenotti–Gerace–Resta [CGR03].

R_f is R_1 reindexed by $\sigma = (AD)(HV)$.

Substitution. An admissible table S of arity q assigns to each letter a set of words $T(a) \subseteq \{B, \dots, V\}^q$. A node $S(c_1, \dots, c_q)$ (`Realisation.multiSubst`) applied to children of exponents e_i is a realisation in $G_4^{\boxtimes \sum e_i}$, with $w_a = \sum_{x \in T(a)} \prod_i w_{x_i}^{(i)}$. We use only BPZ's tables S2a (20 words), S3a (58), S3e (53), S3f (47) and S3g (47).

Terminal code. BPZ's 57-word code K4b, applied to four nodes, gives an independent set of size $M = \sum_{x \in K4b} \prod_{i=1}^4 w_{x_i}^{(i)}$ in $G_4^{\boxtimes E}$, with $E = \sum e_i$ (`le_indepNum_multiCode`). So $p = 4E$.

Tails. $J \times [y \leftarrow S3g(y, R_1, R_1)]$ denotes the J -fold iteration from a start node. Each step raises the exponent by 2.

Schedules (DAGs; a shared sub-schedule is computed once).

- C_{15} (`CertC15b`) has 20 nodes. A tower of S3f/S3e nodes reaches exponent 27. One S2a(R_1, R_f) node is used. Four tails, of lengths 63, 3, 5 and 19, are merged through S3g. K4b is applied to exponents (314, 147, 169, 226), so $E = 856$ and $p = 3424$. BPZ's `CertC15` has 23 nodes and $p = 3664$.
- C_{19} (`CertC19c`) has 26 nodes. An S3f tower reaches exponent 81. Two shoulder nodes, S3a(n_3, n_2, R_1) and S3e(n_3, n_2, R_1), each have exponent 109. Four branches follow. Each branch merges a tail from each shoulder, of lengths (265, 193), (153, 52), (177, 68) and (230, 80), and is closed by S3g. K4b is applied to exponents (1245, 713, 793, 949), so $E = 3700$ and $p = 14800$. BPZ's `CertC19` uses S3f/S3g/S3d and K4b on four exponent-741 nodes, so $p = 11856$.

Appendix A lists both schedules in full. The JSON versions are `schedules/best_C15_A_ref.json` and `best_C19_C_ref.json`.

Lean encoding. A generic file `DagTail.lean` defines `tailIter` by structural recursion and proves `w.tailIter` by induction. It uses ordinary tactics, not `native_decide`, and only BPZ's `multiSubst/w_multiSubst`. Node sizes are computed, never written as literals. The single literal M is checked by one `native_decide (stepM)`.

4. VERIFICATION

- (1) *Patch.* `certL2.patch` (sha256 `ee71e471...ce448e`) applies to `aa21eeb`. It adds five files and five import lines, and changes no BPZ proof. We used Lean 4.32.2 and Mathlib as pinned by BPZ.
- (2) *Axioms.* `#print axioms` lists `propext`, `Classical.choice`, `Quot.sound` and 40 `native_decide` auxiliary axioms, with no `sorryAx`. The only native checks beyond BPZ's own (base data, tables, K4b) are `stepM` and the decimal step. The trust base is the Lean kernel plus compiled evaluation, as for BPZ.
- (3) *Negative control.* With $M + 1$ in place of M , Lean rejects the C_{15} certificate.
- (4) *Independent Python.* A standard-library evaluator, written by a separate agent lane without reading the search or generator code, does the following:
 - it re-parses BPZ's Lean sources and re-checks the tables, codes and port-system fields;
 - it reproduces BPZ's `CertC7`, `CertC15` and `CertC19` exactly, and our two M values digit for digit;
 - it rejects perturbed inputs.
- (5) *Machines.* The check passed on three machines: an Apple M4 Pro laptop, a Mac mini M4 (fresh clone), and a clean-room Ubuntu 24.04 x86_64 desktop with no prior Lean. On the desktop the one-command bundle passed on its first run.
- (6) *Red team.* An adversarial review checked:
 - the statements, with explicit elaboration and the supremum unfolded;
 - the patch scope;
 - for trust holes;
 - the truncation;
 - for dominating limits.

It found no mathematical flaw.

5. REPRODUCIBILITY

```
cd shannon-C15-C19 && sh check.sh --install-elan
```

The script performs these steps:

- checks the bundle hashes;
- clones BPZ at `aa21eeb`;
- applies and audits the patch;
- runs the Python evaluator, with BPZ's certificates as controls;
- builds the certificates;
- re-checks the statements and axioms in a fresh Lean process;
- greps for `sorry`.

It ends with ALL CHECKS PASSED. Research repository: annotated tag `c15-c19-record-v1` on commit `5737f31a6afb08c579c042b5c03991fe3d703057`. The earlier tag `c19-record-v1` (commit `86c62d94`) holds the superseded bound $\Theta(C_{19}) \geq 9.357202700233073$. Public release: <https://github.com/griffinwork40/shannon-capacity-c15-c19>, tag `v1.0` (bundle, tarball with SHA-256, proof sources, logs, reports and this note); `PROVENANCE.md` there maps every released file to the private research repository by commit and hash.

6. HOW THE SCHEDULES WERE FOUND

The search ran inside BPZ's grammar: atoms R_1, R_f , BPZ's tables, J -fold tails, and a terminal BPZ code. It had two stages:

- beam search and simulated annealing, scored in floating point, with total exponent bounded by BPZ's;

- tail-length refinement and exact integer re-evaluation.

Each job took minutes on ordinary machines. The search is not part of the trust base.

Use of AI and software. Griffin Long is the sole author. The computational work, the certificates and the first draft of this note were produced by *agent-afk*, an open-source autonomous agent runtime written by the author (<https://github.com/griffinwork40/agent-afk>). The agent chose the target, wrote the schedule search, the Lean generator and certificates, both checkers and the draft. AI systems are credited here rather than as authors. AI models used through agent-afk (Anthropic):

- Claude Opus, model id `claude-opus-5-5`: coordinator of both working sessions (2026-09-30 and 2026-10-01) and most subagent lanes (baseline reproduction, schedule search, Lean certification, independent and red-team review, novelty search, packaging, drafting);
- Claude Sonnet, model id `claude-sonnet-4-6`: some lanes (an early literature check, the second-machine rerun, finishing an audit report, a repository survey);
- Claude Haiku, model id `claude-haiku-4-5-20251001`: one read-only repository survey.

OpenAI ChatGPT wrote the original task brief given to the agent and, after the result was found, suggested the verification and disclosure plan that was followed; it contributed no mathematical content, code or schedules. Software: Lean 4 (v4.32.2), Mathlib, Lake and elan; Python 3 (standard library only for the checkers; NumPy in exploratory scripts); Google OR-Tools CP-SAT (an earlier, unsuccessful C_7 search); L^AT_EX; Git and the GitHub CLI; Exa web search and the arXiv, Semantic Scholar, OpenAlex, Zenodo and GitHub APIs for the literature search. The author’s inputs, logged in `HUMAN_INPUT.md`, are:

- a goal statement;
- hardware;
- a lane-parallel working style;
- approvals of plans;
- “continue/proceed” messages;
- a forwarded hardening plan;
- the decision to disclose, and review of this text.

No human supplied mathematical content, schedules or code. The search is a plain local search. Its success mainly shows that BPZ’s schedule space was not exhausted.

The literature check covered arXiv, Semantic Scholar, OpenAlex, Zenodo, GitHub, author pages, Wikipedia, MathOverflow and OEIS, up to 2026-10-01. It did not cover Google Scholar, the Lean Zulip (only its web-searchable archive), or unpublished work.

7. REMARKS AND RESIDUAL RISKS

Not new. The framework, tables, codes and Lean infrastructure are BPZ’s, building on Gao and IRCR. The base sets are due to [BMR+71, CGR03, dBBZ24]. No new method is introduced.

Size of gain. In $\log \Theta$ the gains are $8.6 \cdot 10^{-7}$ (C_{15}) and $3.2 \cdot 10^{-7}$ (C_{19}). For comparison, BPZ’s repository improved on their v1 paper by $2.8 \cdot 10^{-5}$ and $7.3 \cdot 10^{-6}$. The Lovász bounds (7.4171 and 9.4348) remain far away. The schedules are not claimed to be optimal. The limiting rates of the tail family $S3g(y, R_1, R_1)$, and of the best homogeneous towers, lie below both BPZ’s values and ours. So the gain comes from finite mixing.

Residual risks.

- (1) The proofs trust `native_decide`, as BPZ’s do.
- (2) Checker hardening (fixed): a hand-declared axiom named like a `native_decide` auxiliary axiom would have passed the original axiom-name filter; `check.sh` now rejects `axiom`, `unsafe`, `opaque`, `implemented_by`, `extern` and macros in the added files.
- (3) Attribution (fixed): the added files now state that they were contributed by Griffin Long with agent-afk as an addition to BPZ’s framework (Apache-2.0), not written by BPZ.

- (4) BPZ have announced improved trees for their next arXiv version. Unpublished work may match or beat these bounds.
- (5) The Python evaluator was written by another instance of the same AI system, and all machines belong to one person. No external party has reproduced the result yet.
- (6) Sixth-decimal improvements from re-ordering known gadgets are of limited interest by themselves. The contribution is certified values and a reproducible mechanised workflow.

Acknowledgements. We thank Pjotr Buys, Sven Polak and Jeroen Zuiddam for releasing their Lean framework openly under the Apache License 2.0. Our patch is a derivative work under the same licence. The use of AI models and software is described in Section 6.

REFERENCES

- [BMR+71] L. D. Baumert, R. J. McEliece, E. Rodemich, H. C. Rumsey Jr., R. Stanley, H. Taylor, A combinatorial packing problem, in: *Computers in Algebra and Number Theory* (Proc. SIAM-AMS Sympos. Appl. Math., New York, 1970), SIAM-AMS Proc., Vol. IV, pp. 97–108, Amer. Math. Soc., Providence, RI, 1971. Zbl 0252.05016.
- [BPZ26] P. Buys, S. Polak, J. Zuiddam, Lean-verified lower bounds for the Shannon capacity of odd cycles, arXiv:2607.29681 (v1, 2026-07-31); Lean repository <https://github.com/spectra-research/shannon-capacity-lean>, commit aa21eeb12b75b0413d3fa9fb4208b5d0bf2c4d65 (2026-08-10).
- [CGR03] B. Codenotti, I. Gerace, G. Resta, Some remarks on the Shannon capacity of odd cycles, *Ars Combin.* **66** (2003), 243–257. Zbl 1073.05544.
- [dBBZ24] D. de Boer, P. Buys, J. Zuiddam, The asymptotic spectrum distance, graph limits, and the Shannon capacity, arXiv:2404.16763 (v1 2024, v2 2026).
- [Gao26] Y. Gao, A recursive construction improving the lower bound on the Shannon capacity of C_7 , arXiv:2607.27869 (2026).
- [IRCR26] N. Itty, C. D. Rosin, C. Carstensen, D. Reichman, Improved lower bounds for the Shannon capacity of odd cycles, arXiv:2607.21517 (v2, 2026).

APPENDIX A. SCHEDULES IN FULL

Exponents are in brackets, in units of $G_4 = C_n^{\boxtimes 4}$.

C_{15} (CertC15b):

n0	= S3f(R1, R1, R1)	[3]
n1	= S3e(R1, R1, R1)	[3]
n2	= S3f(n0, n1, n0)	[9]
n3	= S3f(n0, n0, n0)	[9]
n4	= S3f(n2, n3, n3)	[27]
n5	= S3g(n4, R1, n4)	[55]
n6	= S2a(R1, Rf)	[2]
n7	= S3f(n3, R1, n3)	[19]
n8	= S3g(n5, n6, n7)	[76]
n9	= 63 x [y <- S3g(y, R1, R1)] from y = n5	[181]
n10	= S3g(n5, R1, n9)	[237]
n11	= S3g(n8, R1, n10)	[314]
n12	= 3 x [y <- S3g(y, R1, R1)] from y = n5	[61]
n13	= S3g(n5, R1, n12)	[117]
n14	= 5 x [y <- S3g(y, R1, R1)] from y = n13	[127]
n15	= S3g(n14, n7, R1)	[147]
n16	= 19 x [y <- S3g(y, R1, R1)] from y = n5	[93]
n17	= S3g(n5, R1, n16)	[149]
n18	= S3g(n17, n7, R1)	[169]
n19	= S3g(n8, R1, n17)	[226]
M	= K4b(n11, n15, n18, n19)	E = 856, p = 3424

M has 2957 digits (22412415576865115264...79776353377970664416).

C_{19} (CertC19c):

```

n0 = S3f(R1, R1, R1) [3]
n1 = S3f(n0, n0, n0) [9]
n2 = S3f(n1, n1, n1) [27]
n3 = S3f(n2, n2, n2) [81]
n4 = S3a(n3, n2, R1) [109]
n5 = 265 x [y <- S3g(y, R1, R1)] from y = n4 [639]
n6 = S3e(n3, n2, R1) [109]
n7 = 193 x [y <- S3g(y, R1, R1)] from y = n6 [495]
n8 = S3g(n5, R1, n7) [1135]
n9 = S3g(n8, R1, n3) [1217]
n10 = S3g(n9, R1, n2) [1245]
n11 = 153 x [y <- S3g(y, R1, R1)] from y = n4 [415]
n12 = 52 x [y <- S3g(y, R1, R1)] from y = n6 [213]
n13 = S3g(n11, R1, n12) [629]
n14 = S3g(n13, R1, n3) [711]
n15 = S3g(n14, R1, R1) [713]
n16 = 177 x [y <- S3g(y, R1, R1)] from y = n4 [463]
n17 = 68 x [y <- S3g(y, R1, R1)] from y = n6 [245]
n18 = S3g(n16, R1, n17) [709]
n19 = S3g(n18, R1, n3) [791]
n20 = S3g(n19, R1, R1) [793]
n21 = 230 x [y <- S3g(y, R1, R1)] from y = n4 [569]
n22 = 80 x [y <- S3g(y, R1, R1)] from y = n6 [269]
n23 = S3g(n21, R1, n22) [839]
n24 = S3g(n23, R1, n3) [921]
n25 = S3g(n24, R1, n2) [949]
M = K4b(n10, n15, n20, n25) E = 3700, p = 14800

```

M has 14373 digits (91531227246421702821...35690099930718797824).